Monitor de pulso cardíaco con ESP32, sensor de pulso analógico y pantalla TFT

Cristian Enrique Cruz Fernandez · Samuel Felipe Diaz Galindo · Santiago Reina Socha
Fundación Universitaria de San Gil – UNISANGIL, sede Yopal
Yopal, Casanare, Colombia

***Resumen*—Se presenta el diseño de un monitor de pulso cardíaco educativo basado en un ESP32, un sensor de pulso analógico y una pantalla TFT ST7789. El sensor entrega una tensión que el ADC de 12 bits del microcontrolador muestrea a 100 Hz; el firmware suaviza la señal, elimina la línea base, detecta cada latido con un umbral adaptativo y calcula los BPM con la mediana de los últimos intervalos, validando su regularidad. El resultado se muestra en la pantalla junto con la onda de pulso, un corazón animado en dos tiempos y un aviso sonoro en cada latido, y se transmite al Serial Plotter. Se describen dos versiones del prototipo, el conexionado, el flujo del programa y los bloques del código. A diferencia del montaje con MAX30102, este sistema no mide SpO_2. [COMPLETAR: resultados principales de las pruebas].**

***Palabras clave*—Pulso cardíaco, fotopletismografía, ESP32, sensor analógico, BPM, ST7789.**

I. Introducción

La fotopletismografía (PPG) permite observar cambios periódicos en la luz detectada a medida que varía el volumen sanguíneo periférico: un LED ilumina la yema del dedo y un fotodetector mide la luz que regresa [1]. En este proyecto, un ESP32 lee la señal de un sensor de pulso analógico con su conversor ADC, la procesa y presenta la onda de pulso y la frecuencia cardíaca en una pantalla TFT ST7789, con un corazón animado y un aviso sonoro en cada latido.

El propósito es documentar una implementación reproducible del montaje y explicar cómo se relacionan el sensor, el muestreo, el algoritmo, la interfaz y la salida serie. Respecto al laboratorio anterior con el MAX30102, el sensor analógico simplifica el conexionado y libera el bus I^2C, pero al tener un solo canal no permite estimar SpO_2. El dispositivo es un prototipo educativo: sus resultados dependen del contacto del dedo y de la calidad de la señal, y no se presentan como mediciones clínicas.

II. Materiales

El montaje emplea cuatro módulos principales (Tabla I), ilustrados en las Figs. 1 a 4, además de cables de conexión y un cable USB para alimentación y programación.

A. Tarjeta de desarrollo ESP32

Es el microcontrolador principal. Lee la señal del sensor con el ADC de 12 bits, ejecuta el algoritmo de cálculo, controla el buzzer y envía la información a la pantalla por SPI y al computador por el puerto serie.



Fig. 1. Tarjeta de desarrollo ESP32. elaboración propia

B. Pantalla TFT ST7789

Visualiza la onda de pulso, los BPM, el estado, y el corazón animado. Es un módulo IPS de 240×280 píxeles que se conecta al ESP32 por comunicación SPI.

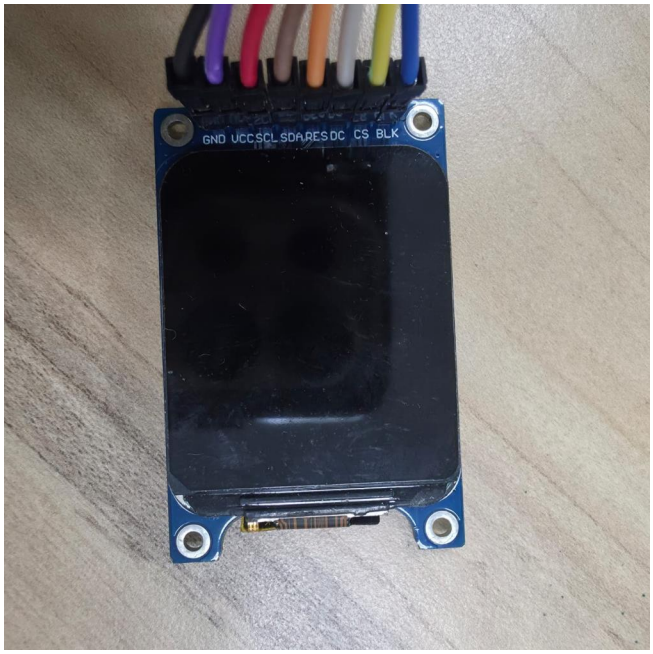


Fig. 2. Módulo TFT ST7789 de 240×280 px. elaboración propia

C. Sensor de pulso analógico

Sensor óptico de tres terminales (positivo, negativo y señal) cuya salida es una tensión proporcional a la luz recibida. Se alimenta con 3,3 V y su señal se lee en el GPIO34 del ESP32. [COMPLETAR: modelo exacto del sensor].

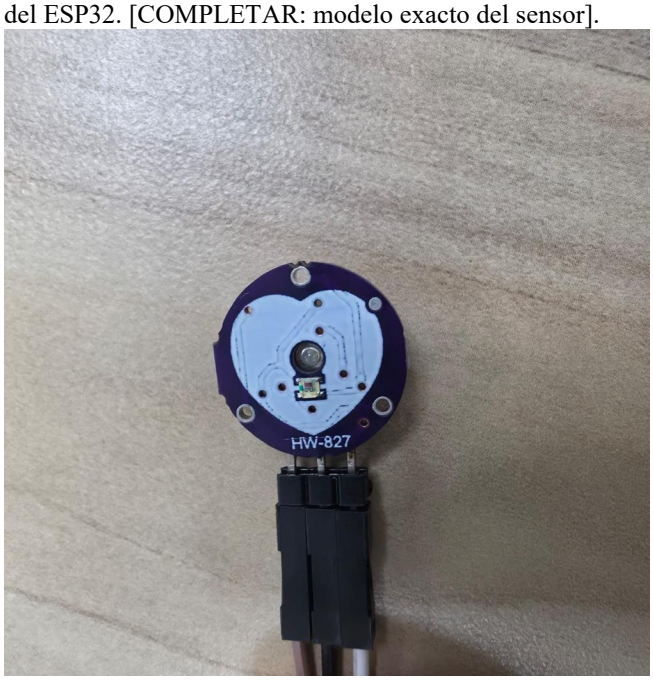


Fig. 3. Sensor de pulso analógico (terminales +, – y S). elaboración propia

D. Buzzer activo

Emite un tono fijo al recibir un nivel alto, sin necesidad de generar una frecuencia desde el programa. Se conecta al GPIO25 y se activa 40 ms en cada latido detectado.



Fig. 4. Buzzer activo. elaboración propia

E. Placa de expansión

Tarjeta donde se inserta el ESP32 y que saca cada pin a una regleta de tres posiciones (G, V y S), lo que permite conectar los módulos con cables sin soldar y de forma ordenada. Se configura a 3,3 V para alimentar el sensor y la pantalla.

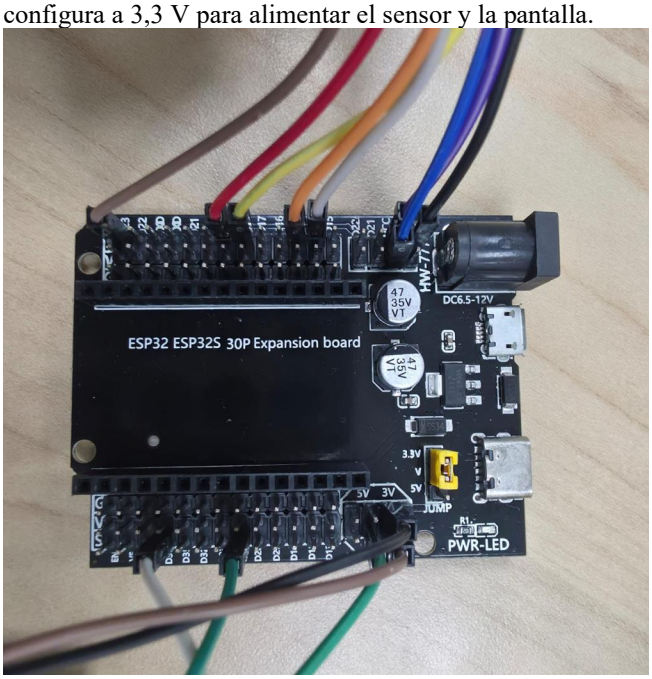


Fig. 5. Placa de expansión. elaboración propia.

F. Cables jumpers

Cables Dupont jumpers. Cables con terminales hembra hembra que unen los módulos con la placa de expansión..



Fig. 6. Cables jumpers. elaboración propia

III. Objetivos

A. Objetivo general

Implementar y describir un sistema con ESP32 y sensor de pulso analógico que adquiera la señal óptica del dedo, estime BPM y muestre la onda, un indicador visual y un aviso sonoro en cada latido, en la pantalla y en el Serial Plotter de Arduino.

B. Objetivos específicos

- Muestrear la señal a 100 Hz y acondicionarla para obtener la componente pulsátil.
- Detectar cada latido y estimar BPM validando la regularidad de los intervalos.
- Visualizar BPM, onda, estado del dedo y corazón animado en la pantalla.
- Avisar cada latido con un buzzer sin bloquear el programa.
- Comparar la versión inicial con la versión completa y verificar la señal en el Serial Plotter.
- Identificar las condiciones de prueba que deben verificarse antes de interpretar las lecturas.

IV. Versiones del prototipo

Durante el desarrollo se implementaron dos versiones del sistema, con el objetivo de mejorar la indicación de cada latido y la lectura de la señal en el Serial Plotter (Tabla II).

A. Versión 1

La primera versión cumple con la adquisición, el filtrado, la detección de latidos y el cálculo de BPM. Muestra el valor en la pantalla ST7789, dibuja la onda y usa un círculo rojo que se enciende 150 ms después de cada latido como único indicador. No tiene buzzer, y el Serial Plotter muestra solo las series Pulso y BPM a 100 datos por segundo, de modo que la ventana, de cantidad fija de puntos, abarca pocos latidos (Fig.5).

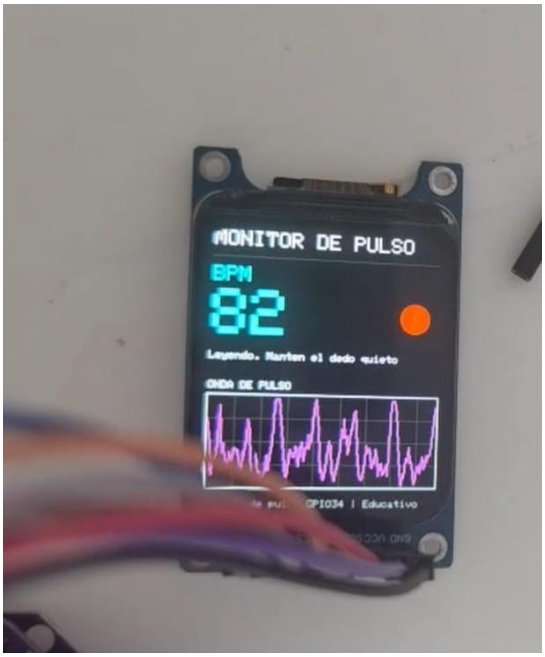


Fig. 5. Versión 1 del prototipo (fuente: elaboración propia).

B. Versión 2

En la segunda versión el círculo se reemplazó por un corazón construido con dos círculos y un triángulo, con un brillo que le da volumen. Con cada latido late en dos tiempos, como el sonido lub-dub: crece durante 80 ms, se contrae hasta los 200 ms, vuelve a crecer a la mitad de amplitud hasta los 250 ms y regresa a su tamaño a los 380 ms. Además, un buzzer pita 40 ms en cada latido, y el Serial Plotter se redujo a 25 datos por segundo con dos series nuevas, Latido y Techo, que facilitan contar las pulsaciones (Fig. 6).

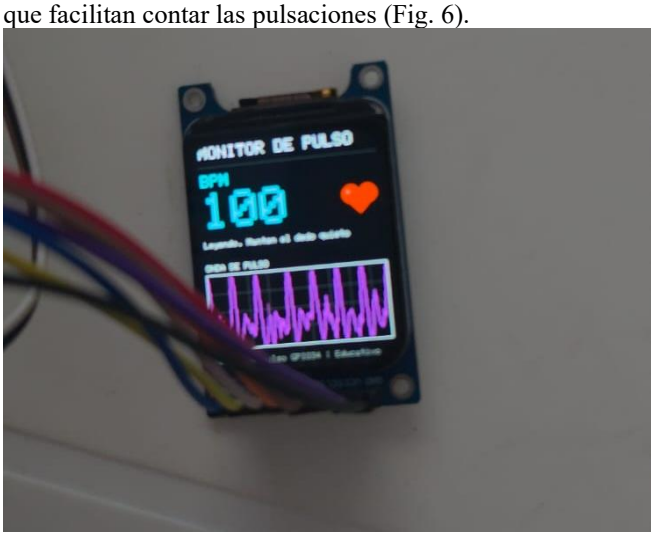


Fig. 6. Versión 2 del prototipo (fuente: elaboración propia).

V. Diagrama de Conexiones

La Fig. 7 muestra el cableado de los módulos y la Tabla III lista cada conexión. El sensor se lee por el ADC en el GPIO34 y la pantalla usa el bus SPI por hardware del ESP32; todos los módulos se alimentan con 3,3 V. El buzzer solo existe en la versión 2.

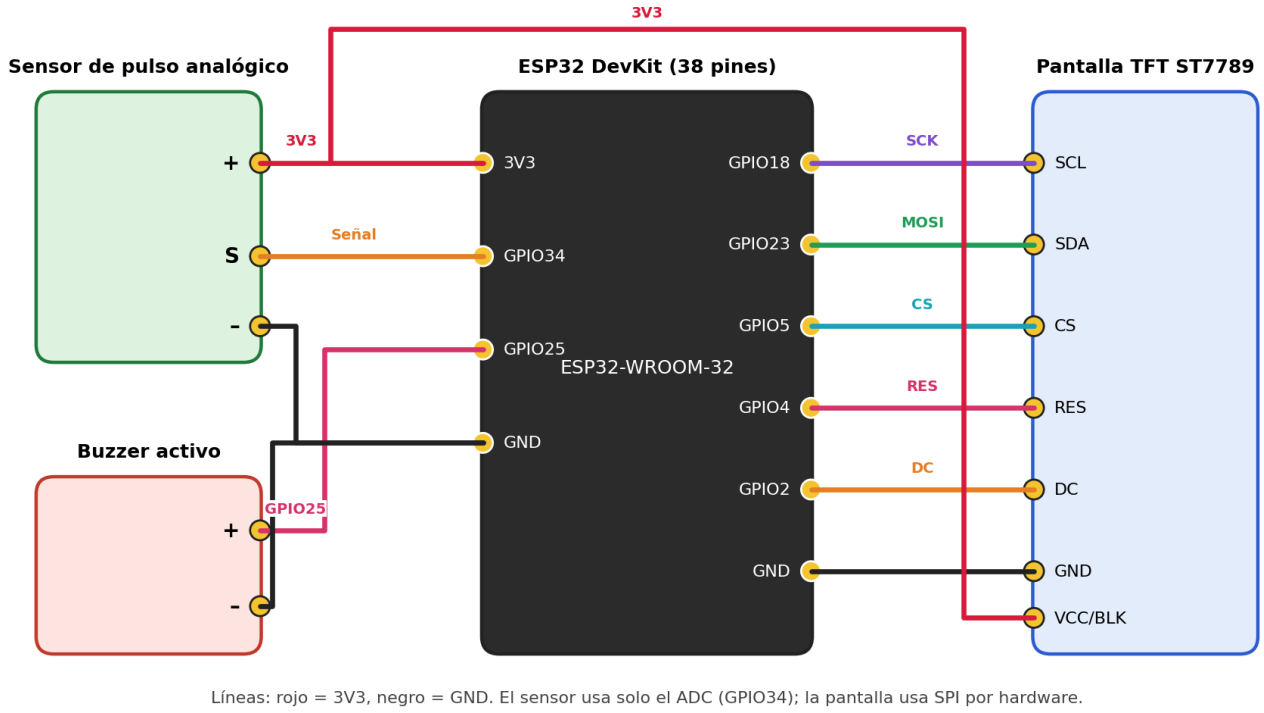


Fig. 7. Diagrama de conexiones entre el ESP32, el sensor, el buzzer y la pantalla ST7789 (fuente: elaboración propia; distribución de pines típica de una ESP32 DevKit de 38 pines).

TABLA III
CONEXIONES ENTRE MÓDULOS Y ESP32

Módulo / señal	Pin ESP32	Interfaz
Sensor +	3,3 V	Alimentación
Sensor S	GPIO 34	ADC
Buzzer +	GPIO 25	Salida digital
Sensor – y buzzer –	GND	Común
TFT SDA / MOSI	GPIO 23	SPI
TFT SCL / SCK	GPIO 18	SPI
TFT CS	GPIO 5	Control
TFT DC	GPIO 2	Control
TFT RES	GPIO 4	Control
TFT VCC / BLK	3,3 V	Alimentación
TFT GND	GND	Común

Notas de conexión

- En la TFT, las etiquetas SDA y SCL corresponden a MOSI y SCK del bus SPI, respectivamente.
- El sensor usa solo el GPIO34, un pin de solo entrada del ADC1; no usa bus I²C.
- El buzzer activo se conecta al GPIO25. Si suena débil, puede manejarse con un transistor NPN y una resistencia de 1 kΩ en la base.
- Los pines VCC y BLK de la pantalla se unen a 3,3 V.
- Confirme el pinout impreso en cada módulo antes de energizar el circuito; el diagrama supone una ESP32 DevKit de 38 pines.

VI. Diagrama de Flujo

La Fig. 8 resume el firmware en cuatro etapas: inicialización (pasos 1 a 3), adquisición (4 a 7), procesamiento (8 a 11) y presentación (12 a 15). Las decisiones detectan la presencia del dedo, el cruce del umbral de latido, la validez del intervalo y la regularidad del pulso.

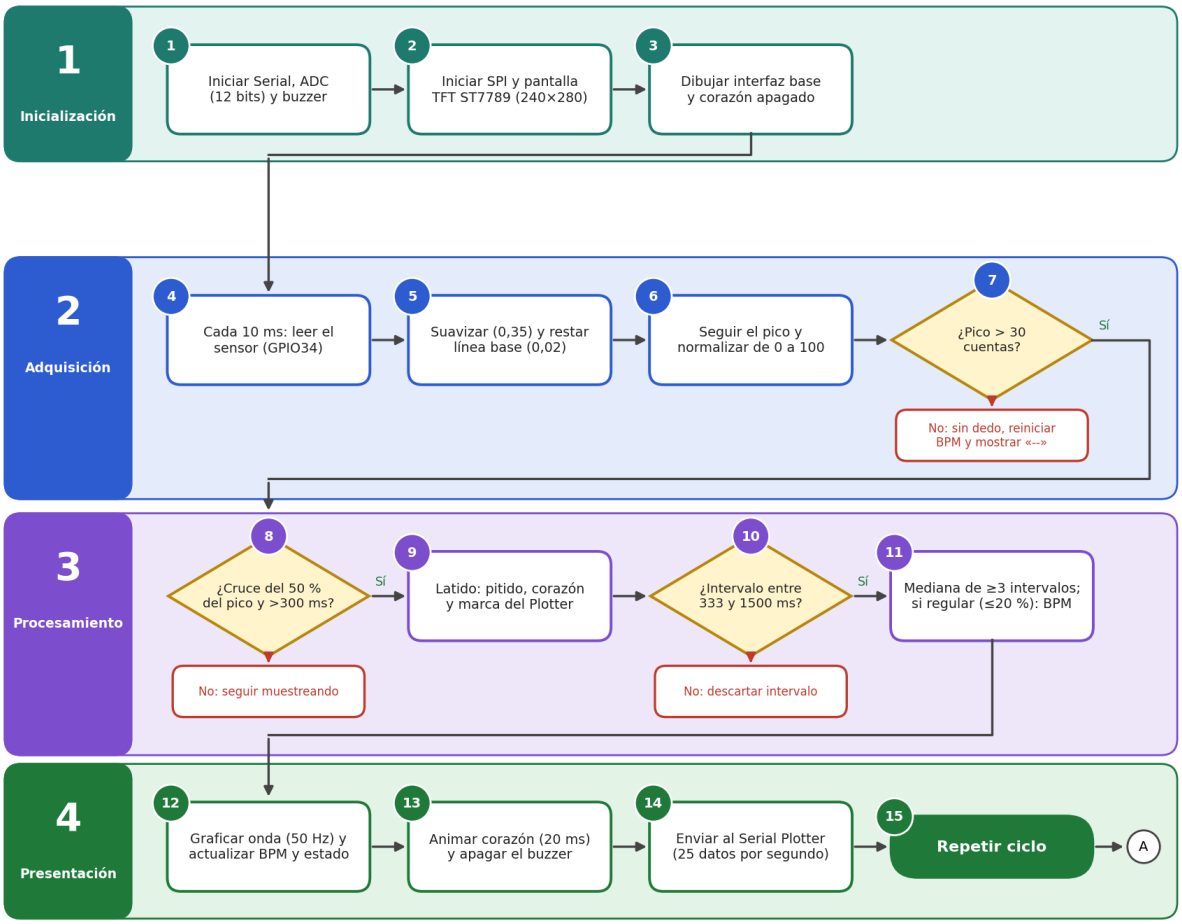


Fig. 8. Diagrama de flujo del firmware, organizado en cuatro etapas y 15 pasos numerados (fuente: elaboración propia).

VII. Configuración y procesamiento

El sensor se muestrea a 100 Hz con el ADC configurado a 12 bits y atenuación de 11 dB. La señal se suaviza con un filtro exponencial (coeficiente 0,35, corte cercano a 7 Hz) y se le resta una línea base lenta (coeficiente 0,02, corte cercano a 0,3 Hz); lo que queda es la componente pulsátil. Un seguimiento del pico de esa componente decide si hay dedo (más de 30 cuentas) y fija el umbral de latido en el 50 % del pico.

Los intervalos entre latidos válidos (333 a 1500 ms) se guardan en un buffer circular de 6 valores. Con al menos 3 se calcula la mediana, y si la desviación media respecto a ella es menor o igual al 20 % se calcula BPM = 60000 / mediana, suavizado con 40 % del valor nuevo y 60 % del anterior. Sin latidos durante 3 s, la lectura caduca y se muestra «--» (Tablas IV y V).

VIII. Código explicado

El firmware completo (Arduino, C++) de la versión 2 se presenta dividido en bloques funcionales. Usa las bibliotecas Adafruit GFX [3] y Adafruit ST7789 [4]; la detección de latidos y el cálculo de BPM son propios y no dependen de bibliotecas del sensor.

Formato de código inspirado en Arduino IDE

Código 1. Encabezado, conexiones y librerías

```
/*
   MONITOR DE PULSO ESP32 + SENSOR DE PULSO ANALOGICO + ST7789 240x280 - v3
   (sin MAX30102; no mide SpO2)

   CONEXIONES (ESP32 clasico, por ejemplo ESP32 Dev Module):
   TFT: SCL=18, SDA=23, RES=4, DC=2, CS=5, VCC/BLK=3.3V, GND=GND.
   SENSOR DE PULSO: Positivo(VCC)=3V3, Negativo(GND)=GND, Senal(S)=GPIO34.
   BUZZER ACTIVO: Positivo=GPIO25, Negativo=GND.

   Bibliotecas: Adafruit GFX Library, Adafruit ST7735 and ST7789 Library.
   Serial: 115200 baudios.

   MODO_PLOTTER=true : Serial Plotter (series: Pulso, BPM, Latido, Techo).
                       BPM=0 = no disponible.
   MODO_PLOTTER=false: Monitor Serie con texto y el valor crudo (Raw).

   Prototipo educativo: no es un dispositivo medico.
*/

```

```
#include <Arduino.h>
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7789.h>
#include <math.h>
#include <stdio.h>
#include <string.h>
```

Código 2. Configuración

```
constexpr int TFT_CS=5, TFT_DC=2, TFT_RST=4;
constexpr int PULSE_PIN=34;
constexpr int BUZZER_PIN=25;
constexpr bool BUZZER_ACTIVO=true;          // false = sin sonido
constexpr uint32_t BIP_MS=40;               // duracion del pitido
constexpr bool MODO_PLOTTER=true;
constexpr uint8_t DIVISOR_PLOTTER=4;        // 100Hz/4 = 25 datos/s (sube a 5 u 8 para ver mas latidos)

constexpr uint32_t PERIODO_MS=10;           // 100 Hz
constexpr uint32_t REFRACTARIO_MS=300;
constexpr uint32_t INTERVALO_MIN_MS=333;    // 180 BPM
constexpr uint32_t INTERVALO_MAX_MS=1500;   // 40 BPM
constexpr uint32_t CADUCIDAD_MS=3000;
constexpr float AMPLITUD_MIN=30.0f;
constexpr int N_INTERVALOS=6;

constexpr int GX=10, GY=164, GW=220, GH=88;
constexpr uint16_t FONDO=ST77XX_BLACK, REJILLA=0x18E3, MAGENTA_A=0xF81F;

// Corazon
constexpr int HX=205, HY=86;                // centro del corazon
constexpr int HR_BASE=8, HR_EXTRA=4;        // radio base y crecimiento
constexpr uint16_t ROJO_VIVO=0xF800;
constexpr uint16_t ROJO_APAGADO=0x4000;
```

Código 3. Objetos, estado global y prototipos

```
Adafruit_ST7789 tft(TFT_CS,TFT_DC,TFT_RST);

uint32_t ultimoMuestreo=0, ultimaUI=0, ultimoTexto=0, ultimaAnim=0;
uint32_t ultimoLatido=0;
uint32_t instanteLatido=0;                  // para animar (no se reinicia a 0)
uint32_t buzzerHasta=0;
bool buzzerEncendido=false;
int rawA=0;
float baseA=0, suaveA=0, bandaA=0, bandaPrevA=0, picoA=0;
float minA=0, maxA=0, onda=0;
bool filtroListo=false, activo=false;
float bpm=-1;
bool bpmValido=false;
uint32_t intervalos[N_INTERVALOS];
int nIntervalos=0, idxIntervalo=0;

uint8_t contPlotter=0;
bool marcaLatido=false;

int xGrafica=0, yAnterior=GY+GH/2;
bool hayPunto=false;
uint8_t divisorGrafica=0;
int bpmPintado=-999;
int radioPintado=-1;
uint16_t colorCorazonPintado=0;
char estadoPintado[40]="";

void dibujarBase();
void limpiarGrafica();
void dibujarColumna(int x);
void graficar(float normalizada);
void muestra();
void reiniciarBPM();
void actualizarUI();
void salidaSerial();
void dibujarCorazon(int r, uint16_t color);
void animarCorazon();
void bip();
void gestionarBuzzer();
```

Código 4. Inicialización y bucle principal

```
void setup() {
  Serial.begin(115200);
  analogReadResolution(12);
  analogSetPinAttenuation(PULSE_PIN,ADC_11db);
  pinMode(PULSE_PIN,INPUT);
  pinMode(BUZZER_PIN,OUTPUT);
  digitalWrite(BUZZER_PIN,LOW);
  SPI.begin(18,-1,23,TFT_CS);
  tft.init(240,280);
  tft.setRotation(0);
  tft.setTextWrap(false);
  dibujarBase();
  actualizarUI();
  animarCorazon();
  ultimoMuestreo=millis();
}

void loop() {
  uint32_t ahora=millis();
  if (ahora-ultimoMuestreo>=PERIODO_MS) {
    ultimoMuestreo=ahora;
    muestra();
  }
  gestionarBuzzer();
  if (ahora-ultimaAnim>=20) {               // animacion del corazon ~50 fps
    ultimaAnim=ahora;
    animarCorazon();
  }
  if (ahora-ultimaUI>=50) {
    ultimaUI=ahora;
    actualizarUI();
  }
  if (!MODO_PLOTTER && ahora-ultimoTexto>=500) {
    ultimoTexto=ahora;
    salidaSerial();
  }
  delay(1);
}
```

Código 5. Buzzer

```
// ---------------- BUZZER ----------------
void bip() {
  if (!BUZZER_ACTIVO) return;
  digitalWrite(BUZZER_PIN,HIGH);
  buzzerEncendido=true;
  buzzerHasta=millis()+BIP_MS;
}

void gestionarBuzzer() {
  if (buzzerEncendido && (int32_t)(millis()-buzzerHasta)>=0) {
    digitalWrite(BUZZER_PIN,LOW);
    buzzerEncendido=false;
  }
```

```
}
```

Código 6. Detección de latidos y cálculo de BPM

```
// ---------------- DETECCION ----------------
void reiniciarBPM() {
  nIntervalos=idxIntervalo=0;
  ultimoLatido=0;
  bpm=-1; bpmValido=false;
}

void muestra() {
  uint32_t ahora=millis();
  rawA=analogRead(PULSE_PIN);
  float x=float(rawA);

  if (!filtroListo) {
    baseA=suaveA=x; minA=x-50; maxA=x+50; picoA=0; bandaPrevA=0;
    filtroListo=true;
  }
  suaveA+=0.35f*(x-suaveA);
  baseA+=0.02f*(suaveA-baseA);
  bandaA=suaveA-baseA;

  float absB=fabsf(bandaA);
  if (absB>picoA) picoA=absB; else picoA+=0.005f*(absB-picoA);

  minA+=0.01f*(suaveA-minA);
  maxA+=0.01f*(suaveA-maxA);
  if (suaveA<minA) minA=suaveA;
  if (suaveA>maxA) maxA=suaveA;
  float amp=fmaxf(maxA-minA,60.0f);
  float centro=(minA+maxA)*0.5f;
  float norm=constrain(0.5f+(suaveA-centro)/amp,0.0f,1.0f);

  activo=(picoA>AMPLITUD_MIN);
  onda=activo ? norm*100.0f : 0.0f;

  float umbral=0.5f*picoA;
  if (activo && bandaPrevA<umbral && bandaA>=umbral &&
      (ahora-ultimoLatido)>REFRACTARIO_MS) {
    instanteLatido=ahora;                    // dispara animacion del corazon
    bip();                                   // dispara el buzzer
    marcaLatido=true;                        // marca el latido para el Plotter
    if (ultimoLatido!=0) {
      uint32_t iv=ahora-ultimoLatido;
      if (iv>=INTERVALO_MIN_MS && iv<=INTERVALO_MAX_MS) {
        intervalos[idxIntervalo]=iv;
        idxIntervalo=(idxIntervalo+1)%N_INTERVALOS;
        if (nIntervalos<N_INTERVALOS) nIntervalos++;
        if (nIntervalos>=3) {
          uint32_t c[N_INTERVALOS];
          memcpy(c,intervalos,sizeof(c));
          for (int i=1;i<nIntervalos;i++) {
            uint32_t k=c[i]; int j=i-1;
            while (j>=0 && c[j]>k) { c[j+1]=c[j]; j--; }
            c[j+1]=k;
          }
          float med=(nIntervalos%2) ? float(c[nIntervalos/2])
                                    : 0.5f*float(c[nIntervalos/2-1]+c[nIntervalos/2]);
          float dev=0;
          for (int i=0;i<nIntervalos;i++) dev+=fabsf(float(c[i])-med);
          dev/=nIntervalos;
          if (dev/med<=0.20f) {
            float b=60000.0f/med;
            bpm=bpmValido ? 0.4f*b+0.6f*bpm : b;
            bpmValido=true;
          } else {
            bpmValido=false; bpm=-1;
          }
        }
      } else if (iv>INTERVALO_MAX_MS) {
        nIntervalos=idxIntervalo=0;
      }
    }
    ultimoLatido=ahora;
  }
  bandaPrevA=bandaA;

  if (!activo) {
    reiniciarBPM();
  } else if (ultimoLatido!=0 && ahora-ultimoLatido>CADUCIDAD_MS) {
    reiniciarBPM();
  }

  if (++divisorGrafica>=2) { divisorGrafica=0; graficar(onda/100.0f); }
  if (MODO_PLOTTER && ++contPlotter>=DIVISOR_PLOTTER) {
    contPlotter=0;
    salidaSerial();
  }
}
```

Código 7. Corazón animado

```
// ---------------- CORAZON ----------------
void dibujarCorazon(int r, uint16_t color) {
  tft.fillRect(HX-26,HY-26,52,54,FONDO); // borra la zona
  int cy=HY-r/2;
  tft.fillCircle(HX-r,cy,r,color); // lobulo izquierdo
  tft.fillCircle(HX+r,cy,r,color); // lobulo derecho
  tft.fillTriangle(HX-2*r,cy,HX+2*r,cy,HY+r+r/2+2,color); // punta
  tft.fillRect(HX-r,cy,2*r,r/2+1,color); // une el centro
  if (color==ROJO_VIVO) { // brillo para dar volumen
    int rb=max(1,r/4);
    tft.fillCircle(HX-r-r/3,cy-r/3,rb,0xFBEF);
  }
}

void animarCorazon() {
  uint32_t t=millis()-instanteLatido;
  float s=0;
  if (activo && instanteLatido!=0) {
    // Latido en dos tiempos: lub (fuerte) y dub (suave)
    if (t<80)        s=t/80.0f;
    else if (t<200)  s=1.0f-(t-80)/120.0f;
    else if (t<250)  s=0.5f*(t-200)/50.0f;
    else if (t<380)  s=0.5f*(1.0f-(t-250)/130.0f);
  }
  int r=HR_BASE+int(lroundf(HR_EXTRA*s));
```

```
    uint16_t color=activo ? ROJO_VIVO :
ROJO_APAGADO;
    if (r!=radioPintado ||
color!=colorCorazonPintado) {
      dibujarCorazon(r,color);
      radioPintado=r;
      colorCorazonPintado=color;
    }
  }
```

Código 8. Interfaz y gráfica

```
// ---------------- PANTALLA ----------------
void dibujarBase() {
  tft.fillScreen(FONDO);
  tft.setTextColor(ST77XX_WHITE);
  tft.setTextSize(2);
  tft.setCursor(12,8); tft.print("MONITOR DE
PULSO");
  tft.drawFastHLine(10,32,220,REJILLA);
  tft.setTextColor(ST77XX_CYAN);
  tft.setCursor(12,42); tft.print("BPM");
  tft.setTextSize(1);
  tft.setTextColor(ST77XX_WHITE);
  tft.setCursor(10,151); tft.print("ONDA DE
PULSO");
  tft.drawRect(GX-1,GY-1,GW+2,GH+2,ST77XX_WHITE);
  tft.setTextColor(0x8410);
  tft.setCursor(10,264); tft.print("Sensor de
pulso GPIO34 | Educativo");
  limpiarGrafica();
}

void dibujarColumna(int x) {
tft.drawFastVLine(GX+x,GY,GH,(x%25==0)?REJILLA:FONDO);
  for (int y=22;y<GH;y+=22)
tft.drawPixel(GX+x,GY+y,REJILLA);
}

void limpiarGrafica() {
  tft.fillRect(GX,GY,GW,GH,FONDO);
  for (int x=0;x<GW;x+=25)
tft.drawFastVLine(GX+x,GY,GH,REJILLA);
  for (int y=22;y<GH;y+=22)
tft.drawFastHLine(GX,GY+y,GW,REJILLA);
  xGrafica=0; yAnterior=GY+GH/2; hayPunto=false;
}

void graficar(float normalizada) {
  int y=GY+GH-3-int(normalizada*(GH-6));
  dibujarColumna(xGrafica);
  dibujarColumna((xGrafica+1)%GW);
  dibujarColumna((xGrafica+2)%GW);
  uint16_t color=activo?MAGENTA_A:REJILLA;
  if (hayPunto && xGrafica>0)
    tft.drawLine(GX+xGrafica-
1,yAnterior,GX+xGrafica,y,color);
  else tft.drawPixel(GX+xGrafica,y,color);
  yAnterior=y; hayPunto=true;
  xGrafica=(xGrafica+1)%GW;
}

void actualizarUI() {
  int b=bpmValido?int(lroundf(bpm)):-1;
  char texto[12];
  if (b!=bpmPintado) {
    if (b<0) strcpy(texto,"-- "); else
snprintf(texto,sizeof(texto),"%-3d",b);
    tft.setTextSize(6);
    tft.setTextColor(ST77XX_CYAN,FONDO);
    tft.setCursor(12,64); tft.print(texto);
    bpmPintado=b;
  }
  char estado[40];
  uint16_t color=ST77XX_WHITE;
  if (!activo) strcpy(estado,"Coloca el dedo
sobre el sensor");
  else if (!bpmValido) {
strcpy(estado,"Midiendo... manten el dedo quieto");
color=ST77XX_YELLOW; }
  else strcpy(estado,"Leyendo. Manten el dedo
quieto");
  if (strcmp(estado,estadoPintado)!=0) {
    tft.fillRect(10,119,220,17,FONDO);
    tft.setTextSize(1); tft.setTextColor(color);
    tft.setCursor(10,123); tft.print(estado);
    strcpy(estadoPintado,estado);
  }
}
```

Código 9. Salida serial

```
// ---------------- SERIAL ----------------
void salidaSerial() {
  char linea[120];
  int b=bpmValido?int(lroundf(bpm)):0;
  int n;
  if (MODO_PLOTTER) {

n=snprintf(linea,sizeof(linea),"Pulso:%.1f,BPM:%d,Lati
do:%d,Techo:130\n",
        onda,b,marcaLatido?100:0);
    marcaLatido=false;                    // se
envia una sola vez
  } else {
    n=snprintf(linea,sizeof(linea),"Raw:%d
Activo:%d Pico:%.0f BPM:%d\n",
        rawA,activo,picoA,b);
  }
  if (n>0 && n<int(sizeof(linea)) &&
Serial.availableForWrite()>=n)
    Serial.write((const uint8_t*)linea,n);
}
```

X. Conclusiones

El sistema integra un ESP32, un sensor de pulso analógico, una pantalla ST7789 y un buzzer para adquirir una señal PPG y mostrar el BPM, la onda, un corazón animado y un aviso sonoro en cada latido. La detección por umbral adaptativo y la mediana de intervalos con validación de regularidad se ejecutan por completo en el microcontrolador. La versión 2 mejora la indicación de cada latido y la lectura en el Serial Plotter respecto a la versión 1, con el mismo procesamiento de señal. [COMPLETAR: conclusiones sobre los resultados de las pruebas].

Se trata de una plataforma académica: hacen falta pruebas físicas documentadas y comparación con un instrumento de referencia antes de valorar la exactitud de las lecturas.

Uso de inteligencia artificial

Para la elaboración de este informe se utilizó el asistente de inteligencia artificial Claude (Anthropic) [7] como apoyo en el desarrollo y la depuración del código, la redacción, la organización y el formato del documento y la elaboración de los diagramas de las Figs. 7 y 8. Los autores revisaron el contenido y asumen la responsabilidad total del trabajo.

REFERENCIAS

[1] J. Allen, “Photoplethysmography and its application in clinical physiological measurement,” Physiological Measurement, vol. 28, no. 3, pp. R1–R39, 2007, doi: 10.1088/0967-3334/28/3/R01.

[2] Espressif Systems, “ESP32 Series Datasheet,” documentación técnica.

[3] Adafruit Industries, “Adafruit GFX Library,” repositorio. [En línea]. Disponible: https://github.com/adafruit/Adafruit-GFX-Library

[4] Adafruit Industries, “Adafruit ST7735 and ST7789 Library,” repositorio. [En línea]. Disponible: https://github.com/adafruit/Adafruit-ST7735-Library

[5] [COMPLETAR: hoja de datos o página del fabricante del sensor de pulso analógico].

[6] [COMPLETAR: fuentes y licencias de las imágenes de las Figs. 1 a 4].

[7] Anthropic, “Claude,” asistente de inteligencia artificial, modelo Claude Sonnet 5.5, 2026. [En línea]. Disponible: https://claude.ai

SITIO WEB DEL PROYECTO

Página web del proyecto, con el video, el código y el informe: [https://github.com/cristiancruzclud/monitor-pulso-esp3202.git].